

PRACTICAL WEEK 5

Task-1

Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

```
imvarls@LAPTOP-VQ4D70L7: × + ▾
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main()
{
    pid_t pid1, pid2, zombie;
    // Create Child 1
    pid1 = fork();
    if (pid1 == 0)
    {
        printf("Child 1: PID = %d, PPID = %d\n", getpid(), getppid());
        exit(0);
    }

    // Create Child 2
    pid2 = fork();
    if (pid2 == 0)
    {
        printf("Child 2: PID = %d, PPID = %d\n", getpid(), getppid());
        exit(0);
    }

    // Synchronization using wait()
    wait(NULL);
    printf("One child completed using wait().\n");

    // Synchronization using waitpid()
    waitpid(pid2, NULL, 0);
    printf("Second child completed using waitpid().\n");

    // Zombie process demonstration
    zombie = fork();

    if (zombie == 0)
    {
        printf("Zombie Child (PID = %d) exiting...\n", getpid());
        exit(0);
    }
    else
    {
        printf("\nParent sleeping for 10 seconds.\n");
        printf("Run: ps -el | grep q4\n");
        sleep(10);          // Child becomes Zombie

        wait(NULL);         // Remove Zombie
        printf("Zombie removed successfully.\n");
    }

    return 0;
}
```

```
imvarls@LAPTOP-VQ4D70L7:~$ nano week4.c
imvarls@LAPTOP-VQ4D70L7:~$ gcc week4.c -o week4
imvarls@LAPTOP-VQ4D70L7:~$ ./week4
Child 1: PID = 608, PPID = 607
Child 2: PID = 609, PPID = 607
One child completed using wait().
Second child completed using waitpid().

Parent sleeping for 10 seconds.
Run: ps -el | grep q4
Zombie Child (PID = 610) exiting...
Zombie removed successfully.
imvarls@LAPTOP-VQ4D70L7:~$ |
```

Observation

- An anonymous pipe was successfully created for communication between the parent and child processes.
- The parent process acted as the producer and generated 1000 integer values.
- The child process acted as the consumer and read the data from the pipe.

- The parent and child communicated successfully without using a shared file.
- The communication time was measured using `clock_gettime()`.
- Communication throughput was calculated in bytes per second, providing a measure of communication efficiency.
- The parent used `wait()` to synchronize with the child and ensure that the consumer completed before the parent terminated.
- Thus, anonymous pipes provide a simple and efficient one-way inter-process communication (IPC) mechanism between related processes.

Task -2 Develop a program that executes the equivalent of the shell command: `ls -l grep` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

```
imvaris@LAPTOP-VQ4D70L7: × + v
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int pipefd[2];
    pid_t pid1, pid2;

    // Create the pipe
    if (pipe(pipefd) == -1) {
        perror("pipe failed");
        exit(1);
    }

    // Create first child for 'ls -l'
    pid1 = fork();
    if (pid1 < 0) {
        perror("fork failed");
        exit(1);
    }

    if (pid1 == 0) {
        // Child 1: Redirect stdout to write end of pipe
        close(pipefd[0]); // Close read end
        dup2(pipefd[1], STDOUT_FILENO);
        close(pipefd[1]);

        execlp("ls", "ls", "-l", NULL);
        perror("execlp ls failed");
        exit(1);
    }

    // Create second child for 'grep .c'
    pid2 = fork();
    if (pid2 < 0) {
        perror("fork failed");
        exit(1);
    }

    if (pid2 == 0) {
        // Child 2: Redirect stdin from read end of pipe
        close(pipefd[1]); // Close write end
        dup2(pipefd[0], STDIN_FILENO);
        close(pipefd[0]);

        execlp("grep", "grep", ".c", NULL);
        perror("execlp grep failed");
        exit(1);
    }

    // Parent process closes both ends of pipe
    close(pipefd[0]);
    close(pipefd[1]);

    // Wait for both children to terminate
    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);

    printf("Pipeline execution completed.\n");
}
```

```
// Create second child for `grep .c`
pid2 = fork();
if (pid2 < 0) {
    perror("fork failed");
    exit(1);
}

if (pid2 == 0) {
    // Child 2: Redirect stdin from read end of pipe
    close(pipefd[1]); // Close write end
    dup2(pipefd[0], STDIN_FILENO);
    close(pipefd[0]);

    execlp("grep", "grep", ".c", NULL);
    perror("execlp grep failed");
    exit(1);
}

// Parent process closes both ends of pipe
close(pipefd[0]);
close(pipefd[1]);

// Wait for both children to terminate
waitpid(pid1, NULL, 0);
waitpid(pid2, NULL, 0);

printf("Pipeline execution completed.\n");

return 0;
}
```

```

imvarls@LAPTOP-VQ4D70L7:~$ nano week5task2.c
imvarls@LAPTOP-VQ4D70L7:~$ gcc week5task2.c -o weektask2
imvarls@LAPTOP-VQ4D70L7:~$ ./week5task2
-bash: ./week5task2: No such file or directory
imvarls@LAPTOP-VQ4D70L7:~$ nano week5task2.c
imvarls@LAPTOP-VQ4D70L7:~$ gcc week5task2.c -o week5task2
imvarls@LAPTOP-VQ4D70L7:~$ ./week5task2
-rw-r--r-- 1 imvarls imvarls 1308 Aug  4 07:11 File1.txt.c
-rwxr-xr-x 1 imvarls imvarls 16216 Sep 11 09:22 client
-rw-r--r-- 1 imvarls imvarls 719 Sep 11 09:20 client.c
-rwxr-xr-x 1 imvarls imvarls 16200 Aug  4 15:20 copy
-rw-r--r-- 1 imvarls imvarls 820 Aug  4 15:20 copy.c
-rw-r--r-- 1 imvarls imvarls 1529 Aug  4 14:53 copy_file.c
-rw-r--r-- 1 imvarls imvarls 473 Aug  8 03:57 demo.c
-rw-r--r-- 1 imvarls imvarls 2251 Aug 11 06:39 file.c
-rw-r--r-- 1 imvarls imvarls 1536 Aug  8 03:24 file2.c
-rw-r--r-- 1 imvarls imvarls 944 Aug  4 15:01 filename.c
-rw-r--r-- 1 imvarls imvarls 6 Jul 25 04:19 filename.c.save
-rw-r--r-- 1 imvarls imvarls 23 Jul 28 06:54 first.c.save
-rw-r--r-- 1 imvarls imvarls 144 Jul 28 07:09 first.c.save.1
-rw-r--r-- 1 imvarls imvarls 5896 Aug 11 15:18 myshell.c
-rwxr-xr-x 1 imvarls imvarls 16344 Sep 12 02:51 process
-rw-r--r-- 1 imvarls imvarls 1399 Sep 12 02:51 process.c
-rwxr-xr-x 1 imvarls imvarls 16296 Aug  8 03:34 process_state
-rw-r--r-- 1 imvarls imvarls 1536 Aug  8 03:34 process_state.c
-rw-r--r-- 1 imvarls imvarls 1536 Aug  8 03:18 processtate.c
-rw-r--r-- 1 imvarls imvarls 719 Sep 11 09:20 server.c
-rw-r--r-- 1 imvarls imvarls 1308 Aug  4 07:05 task2.c
-rw-r--r-- 1 imvarls imvarls 680 Aug 21 13:53 week1.c
-rw-r--r-- 1 imvarls imvarls 990 Sep 12 03:27 week4.c
-rw-r--r-- 1 imvarls imvarls 2064 Sep 12 03:24 week5.c
-rw-r--r-- 1 imvarls imvarls 1352 Sep 12 03:35 week5task2.c
-rw-r--r-- 1 imvarls imvarls 706 Sep 12 03:01 zombie.c
Pipeline execution completed.
imvarls@LAPTOP-VQ4D70L7:~$ |

```

Observation

- A pipe was created using the `pipe()` system call to establish communication between two child processes.
- The first child used `dup2()` to redirect its standard output to the pipe and executed `ls -l` using `exec()`.
- The second child redirected its standard input from the pipe using `dup2()` and executed `grep .c` using `exec()`.

- The output produced by `ls -l` was therefore passed directly to `grep`, just like a normal shell pipeline.
- The parent process closed the unused pipe ends and used `waitpid()` to synchronize with both child processes.
- The program successfully demonstrated IPC using pipes, file-descriptor redirection using `dup2()`, process creation using `fork()`, and command execution using `exec()`.